

Table of Contents

INTRODUCTION	2
PROJECT IDEA	2
MOTIVATION.....	2
DOMAIN AND INTENDED USERS.....	2
PROJECT GOALS, OBJECTIVES, AND FRAMEWORK.....	3
LITERATURE REVIEW	4
REINFORCEMENT LEARNING IN FINANCIAL TRADING	5
MOVING AVERAGE-BASED STRATEGIES IN TECHNICAL TRADING	5
BACKTESTING METHODOLOGIES AND STRATEGY VALIDATION	7
EXPLAINABLE AI AND NATURAL LANGUAGE GENERATION IN FINANCE	8
EXISTING STOCK ADVISOR BOTS AND FINTECH TOOLS ...	9
UNIQUE SELLING PROPOSITION (USP)	10
DESIGN.....	11
PROJECT OVERVIEW AND JUSTIFICATION	11
HIGH-LEVEL SYSTEM ARCHITECTURE.....	12
SEQUENCE DIAGRAM	1
INTERACTION AND UX DESIGN.....	1
KEY COMPONENTS AND THEIR RESPONSIBILITIES.....	1
TECHNOLOGY STACK.....	3
WORK PLAN & GANTT CHART	3
EVALUATION PLAN	1
IMPLEMENTATION	1
STRATEGY LOGIC	1
EXPLANATION ENGINE	3
EVALUATION ENGINE	4
OUTPUT AND METRICS	4
WEB INTERFACE	5
EVALUATION.....	6
OVERVIEW AND EVALUATION STRATEGY	6
1. TECHNICAL EVALUATION	6
2. EXPLAINABILITY AND USER EVALUATION.....	7
3. CRITICAL ANALYSIS	8
4. FUTURE ENHANCEMENTS.....	8
5. EVALUATION SUMMARY	8
CONCLUSION.....	8

SUMMARY OF KEY ACHIEVEMENTS.....	9
REFERENCES	11

Introduction

The current financial market with its unstable conditions and abundant data requires investors who lack technical skills and retail investors to make fast and accurate investment choices. The fast development of artificial intelligence (AI) together with changing market conditions has created an expanding difference between sophisticated trading systems and typical investor capabilities. The Financial Advisor Bot project solves this problem by creating an intelligent system which provides explainable investment advice through a user-friendly interface. The bot provides individualized investment recommendations through its simple web-based interface which translates complex algorithms into easy-to-understand language for users.

Project Idea

The Financial Advisor Bot operates on a core principle to create an interactive platform which processes real-time stock market information to produce investment recommendations for specific stock symbols through BUY, SELL or HOLD signals. The system produces its recommendations by uniting Reinforcement Learning (RL) algorithms with moving average crossovers and other technical indicators. The bot provides natural language explanations to support its recommendations which help non-technical users understand the basis of each investment decision. The system functions as a digital version of human financial advisors who provide strategic guidance and explain their decisions to users instead of showing uninterpretable data or unclear predictions.

Motivation

Expert analysts together with institutional investors who possess premium datasets and advanced analytical

tools and market experience have controlled financial decision-making for many years. Most retail investors face resource constraints because they depend on their instincts and basic trading platforms and online discussions for their investment decisions. The market has experienced an increased power imbalance because AI trading bots and high-frequency strategies used by hedge funds enable them to detect small price changes at speeds beyond human capabilities.

The main objective of this research initiative focuses on creating equal access to intelligent investment methods which will enable people without financial or technical expertise to achieve better investment decisions. The current market offers multiple tools which enable users to monitor prices and study charts but these tools lack the ability to generate explainable customized investment strategies that users can understand easily. The project unites artificial intelligence analytical capabilities with user-friendly design elements and simple communication methods to achieve its goal.

The increasing popularity of educational investing requires systems which provide advice through explanations to users who want to learn during their investment process. The Financial Advisor Bot fulfills the current market requirement by delivering instant financial insights through simple language which functions as a digital advisor and educational resource.

Domain and Intended Users

The Financial Advisor Bot operates within the stock market advisory domain, focusing primarily on U.S.-listed equities such as AAPL, TSLA, and NFLX. The stock market provides an ideal testing ground for this system because it offers abundant historical data, fixed trading sessions, and a well-regulated structure that simplifies model validation. In contrast, markets like

cryptocurrency or foreign exchange - though dynamic - demand complex infrastructure to manage their continuous, high-volatility nature.

By analysing daily stock price data and applying technical indicators such as moving averages, the bot is designed to simulate short- to medium-term trading behaviours that mirror those of everyday investors. The inclusion of historical backtesting allows measurable evaluation of strategy performance, ensuring that outcomes are not only interpretable but also empirically verifiable. Moreover, the cyclical and news-sensitive nature of equity markets makes this domain particularly suitable for reinforcement learning (RL), where decision-making under uncertainty is central.

The system's intended users are primarily non-technical retail investors - individuals who may lack expertise in algorithmic trading, financial modelling, or machine learning. These users typically rely on basic brokerage applications or online resources and often face difficulty interpreting conflicting information. The Financial Advisor Bot aims to empower them by delivering clear, actionable, and well-explained recommendations that simplify complex trading logic into human-understandable insights. Beyond retail investors, the bot also serves students and educators in finance, business, and computer science who wish to explore the intersection of AI and investment strategy. It is equally valuable for beginner traders seeking guidance and validation for their decisions, as well as for data science researchers or developers studying human-AI collaboration in practical financial contexts.

Through this combination of a well-structured domain and an inclusive user base, the Financial Advisor Bot bridges the gap between sophisticated algorithmic systems and real-world accessibility - bringing intelligent, explainable investing within reach of everyone.

Project Goals, Objectives, and Framework

The overarching goal of the Financial Advisor Bot is to design, develop, and evaluate a fully functional advisory system that operates intelligently while communicating in a clear and user-friendly manner. The system's objectives are to:

- Fetch and preprocess stock market data using external APIs such as Yahoo Finance.
- Apply a hybrid analytical approach that combines technical indicators with a Q-learning-based reinforcement learning agent to generate investment signals.
- Evaluate strategy performance through historical backtesting, including visualisation of equity curves and detailed trade summaries.
- Translate complex outputs into actionable BUY, SELL, or HOLD recommendations.
- Generate natural language explanations tailored to each user's understanding level.
- Present all results within a modern, animated, and responsive web interface designed for intuitive interaction and engagement.

These objectives align with the broader goals of technical reliability, usability, and explainability, ensuring that the final system is not only functional but also accessible and trustworthy for end users.

To achieve these aims, the project adopts the Decision Making Under Uncertainty framework - a model that mirrors the real-world conditions of financial investing, where complete information is rarely available. At its foundation lies Reinforcement Learning (RL), which enables the system to learn from experience by making sequential decisions and improving through feedback.

In practice, this framework is implemented through:

- A Q-learning algorithm that trains the agent using simulated trading episodes, rewarding profitable actions and penalising poor ones.
- The integration of moving average crossover strategies, adding interpretability and validation to RL-based signals.
- A natural language explanation engine, optionally enhanced with local language models such as Ollama, to generate fluent, context-aware justifications.
- A comprehensive evaluation pipeline that measures performance through metrics such as profit/loss ratios, trade accuracy, and equity curve visualisations.

Together, these components unify AI methodologies, software engineering principles, and user interface design into a seamless system that provides not only predictions but transparent, meaningful, and educational investment guidance.

Literature review

Over the past decade, the field of automated financial advising has undergone a remarkable transformation driven by advances in machine learning, reinforcement learning, and explainable artificial intelligence (XAI). These technologies have revolutionised how investment decisions are analysed, optimised, and executed. Algorithmic trading systems, in particular, have become dominant players in global financial markets, offering levels of speed, consistency, and analytical precision that far exceed human capability. While such systems demonstrate exceptional technical performance, they are primarily built for institutional investors or individuals with substantial expertise in finance, statistics, and data science. Consequently, a significant portion of retail investors - those who seek financial

guidance without deep technical knowledge - find these tools complex and inaccessible.

This gap underscores an urgent and growing demand for intuitive, transparent, and user-centred financial advisory systems that bridge the divide between advanced computational intelligence and real-world user needs. The ideal system would merge the predictive power of algorithmic models with interfaces that support comprehension, trust, and informed decision-making.

The Financial Advisor Bot project is positioned precisely at this intersection. It integrates several complementary areas of research: reinforcement learning for decision-making under uncertainty, technical analysis based on moving averages, strategy backtesting for model validation, and natural language generation for explainability and interpretability. Beyond these technical layers, the project critically examines how existing advisory bots often fall short in transparency, adaptability, and usability for non-expert users.

Accordingly, this literature review synthesises existing academic and industrial contributions across these domains, identifies the persistent gaps in current solutions, and situates the present project as a hybrid, explainable approach tailored to empower retail investors. Each subsection draws on foundational studies, open-source research tools, and recent innovations to justify the architectural and design choices that underpin the system.

Reinforcement Learning in Financial Trading

Reinforcement Learning (RL) has gained substantial attention in the financial domain, as its capacity for sequential and adaptive decision-making mirrors the uncertain dynamics of trading environments. Within this framework, algorithms learn optimal strategies through iterative interaction with the market, continuously refining actions based on reward-based feedback. Among these, Q-learning has emerged as a popular approach for modelling trading agents capable of adapting to changing market conditions.

An early application by Li et al. (2009) employed a tabular Q-learning model for stock trading and demonstrated that adaptive RL agents could outperform traditional buy-and-hold strategies, particularly during periods of heightened volatility. However, their implementation was largely experimental and lacked features crucial for practical deployment, such as real-time decision capability, explainability, and integration with established technical indicators. Consequently, while effective in controlled simulations, it offered limited value for end-user financial systems.

Building on this foundation, Moody and Saffell (2001) introduced a recurrent reinforcement learning model aimed at optimising the Sharpe ratio in dynamic portfolio management. By embedding memory of past performance into the learning process, their approach enhanced long-term adaptability and profitability. Nonetheless, the technique's reliance on mathematically complex constructs - such as logarithmic utility functions - reduced its transparency, making it less accessible to non-specialist users or retail investors seeking intuitive insights.

A major advancement came from Jiang et al. (2017), who applied Deep Q-Networks (DQN) to

cryptocurrency markets. Their model leveraged deep neural architectures to automatically extract patterns from raw market data, removing the need for handcrafted features. While the results showed improved accuracy and responsiveness, the deep RL framework introduced notable drawbacks: unstable training, high computational demands, and minimal interpretability. These limitations restrict its practicality for lightweight, transparent retail applications.

In contrast, the Financial Advisor Bot adopts a classical Q-learning approach designed to balance computational efficiency, interpretability, and performance. By integrating RL with clear, rule-based technical indicators, the system maintains user trust while enabling adaptive market responses. Unlike the opaque, resource-heavy models of Jiang et al. (2017) or the mathematically abstract methods of Moody and Saffell (2001), this bot provides an explainable decision flow supported by interactive user feedback. Its modular architecture also ensures scalability, allowing seamless future upgrades to more complex RL frameworks without disrupting usability. Through this balance, the project preserves the strengths of reinforcement learning - adaptivity, sequential reasoning, and market sensitivity - while addressing the enduring challenge of explainability in AI-driven finance.

Moving Average-Based Strategies in Technical Trading

Moving Average (MA) strategies represent one of the most enduring and widely applied techniques in technical analysis, valued for their clarity, interpretability, and practical efficiency in identifying market trends. The principle behind them is straightforward: smoothing price data over a specific period to highlight underlying direction and reduce market noise. Among the many variations, MA

crossover strategies are especially popular. In these systems, a BUY signal is triggered when a short-term MA (e.g., 10-day) crosses above a longer-term MA (e.g., 50-day), suggesting upward momentum, while a SELL signal occurs when the short-term average dips below the long-term one, signalling a potential downturn.

A seminal contribution by Brock, Lakonishok, and LeBaron (1992) provided empirical evidence supporting the predictive power of such strategies. Analysing decades of Dow Jones Industrial Average data, they demonstrated that even simple crossover rules like the 50-day/200-day combination could outperform random-walk benchmarks. This study was pivotal in legitimising technical analysis within academic finance and remains a reference point for subsequent algorithmic trading research. Nevertheless, their work was limited by its static design—it lacked adaptive mechanisms that respond to shifting market regimes and offered no direct application to real-time or user-interactive systems. The fixed nature of these strategies made them reliable for historical testing but less practical for modern, dynamic markets, where conditions evolve rapidly.

Expanding on these foundations, Neely et al. (2009) investigated MA-based strategies within currency markets. Their research confirmed that technical trading rules - including those centred on moving averages - can generate excess returns when transaction costs are minimal. Yet, these models were built on assumed historical stability, operating under the notion that relationships among variables would remain constant over time. Such an assumption overlooks the adaptability required in volatile financial environments. Moreover, these approaches lacked integration with interactive, learning-based frameworks and offered little room for user interpretability or engagement,

keeping them largely confined to the domain of quantitative analysts.

As computational power and data availability improved, researchers began merging machine learning with traditional technical indicators to enhance predictive performance. For example, Patel et al. (2015) incorporated moving-average signals as part of an ensemble model combining multiple classifiers. This hybrid structure improved forecast precision but came at the cost of increased model opacity and data requirements. The resulting systems demanded high-dimensional inputs and produced outcomes that were difficult to explain - especially to non-expert users unfamiliar with algorithmic reasoning.

The Financial Advisor Bot builds upon these insights by adopting a hybrid methodology that maintains the transparency of classic MA strategies while integrating them within a reinforcement-learning framework. Specifically, the bot computes 10-day and 50-day moving averages to detect short-term and medium-term trend reversals. These indicators serve both as independent trading signals and as contextual inputs for the Q-learning agent, allowing it to adapt decisions based on evolving market patterns. What differentiates this project from prior studies is its explainability layer - each signal is accompanied by a natural-language explanation that articulates why a particular recommendation is made.

By translating quantitative reasoning into understandable language, the system empowers retail users to follow not just the what but also the why behind every decision. This combination of analytical rigour, transparency, and human-centred communication ensures that the simplicity of MA strategies is preserved while their adaptability and usability are enhanced. In doing so, the Financial Advisor Bot advances beyond earlier static or opaque approaches, aligning its

technical core with the broader goal of accessible, educational, and trustworthy AI-driven financial advice.

Backtesting Methodologies and Strategy

Validation

Backtesting serves as a fundamental pillar of algorithmic trading, providing a systematic way to evaluate whether a trading strategy can perform effectively under historical market conditions. By simulating trades on past data, developers and analysts can assess not only the profitability of a given algorithm but also its robustness, stability, and resilience in the face of volatility. In essence, backtesting bridges the gap between theoretical model design and practical market performance, ensuring that trading strategies are both conceptually valid and operationally sound.

One of the earliest and most influential contributions to this area came from Lo, Mamaysky, and Wang (2000), who developed a pattern recognition-based backtesting framework for validating technical trading signals. Their model combined signal-processing techniques with structured trading rules and evaluated performance through historical return analysis. The approach was methodologically rigorous and provided a quantitative foundation for linking market patterns with trading outcomes. However, it lacked real-time feedback mechanisms and was not designed for user interaction or adaptability, limiting its usefulness beyond controlled research environments.

In modern practice, developers increasingly turn to open-source frameworks such as Backtrader (2023) and QuantConnect (2024) to implement and test algorithmic strategies. Backtrader offers a Python-based environment that supports detailed logging, brokerage simulation, and comprehensive risk control features, making it a powerful tool for professional developers.

Despite its flexibility, it remains largely geared toward experienced traders and lacks seamless integration with web-based or consumer-oriented platforms. QuantConnect, by contrast, provides an advanced, cloud-hosted infrastructure capable of running complex strategies across multiple asset classes. However, its dependence on registration, internet connectivity, and server-side setup reduces accessibility for developers seeking lightweight, standalone implementations suitable for educational or individual use.

The Financial Advisor Bot takes a different route by introducing a custom-built backtesting module specifically tailored to the project's goals of accessibility and transparency. This internal module computes essential performance metrics such as total return, number of trades, win/loss ratio, and average profit or loss per trade, enabling users to evaluate both the profitability and efficiency of the system. It also generates an equity curve, visually representing portfolio growth over time. This dual-layer feedback - quantitative through metrics and qualitative through visualisation - enhances user comprehension and engagement, turning complex performance data into an interpretable narrative of the trading journey.

Another defining feature of this module is its trade simulation mechanism, which mirrors realistic trading constraints such as cash availability, share quantity, and transaction sequence. This ensures that backtesting outcomes more accurately reflect the behaviour of the reinforcement learning (RL) strategy in live market conditions. Each trade event is meticulously logged, supporting both transparency and post-analysis, while offering a valuable educational component for users learning how algorithmic systems make decisions.

By integrating these functionalities, the backtester effectively fills a usability gap left by traditional academic prototypes and heavyweight open-source

frameworks. It combines the rigour of professional-grade validation tools with the simplicity of a user-friendly interface. Moreover, the module supports real-time interaction, allowing users to experiment with different stock symbols and immediately visualise performance summaries. This interactive experience transforms backtesting from a purely analytical process into a hands-on learning tool, reinforcing the project's mission to democratise financial intelligence through explainable and accessible AI systems.

Explainable AI and Natural Language Generation in Finance

As financial systems increasingly rely on complex algorithms to make and justify decisions, the need for transparency, interpretability, and user trust has become paramount. In domains where decisions directly affect monetary outcomes, risk exposure, and investor confidence, the ability to understand why a system recommends a particular action is as critical as the recommendation itself. This has led to a growing emphasis on Explainable Artificial Intelligence (XAI) - a branch of AI that seeks to translate opaque computational logic into human-understandable reasoning. In financial contexts, where human trust and algorithmic precision intersect, explainability is not merely a desirable feature but a fundamental requirement, particularly for retail investors who may lack deep technical or financial expertise.

One of the earliest demonstrations of XAI in finance-oriented systems was by Krause et al. (2016), who integrated explainability into fraud detection workflows. Their study introduced a template-based explanation framework, transforming classification outputs into concise, human-readable narratives. This innovation significantly enhanced user trust and perceived system credibility, showing that transparency

could meaningfully influence adoption. However, their approach relied heavily on static templates that produced repetitive, non-adaptive explanations. Because these templates did not respond to contextual variations in input data or user behaviour, their applicability to dynamic financial environments remained limited.

The emergence of Large Language Models (LLMs) has expanded the boundaries of explainable communication. Chen et al. (2021) explored the use of GPT-style generative models to convert quantitative market signals into fluent, human-like investment summaries. While these systems demonstrated impressive linguistic capability and natural tone, they faced substantial challenges in domain accuracy, hallucination risks, and computational expense. Their dependence on cloud-based APIs also raised legitimate concerns about data privacy, latency, and cost-efficiency, particularly in real-time financial advisory settings where responsiveness and reliability are essential.

Recently, researchers have pursued hybrid Natural Language Generation (NLG) frameworks that balance consistency with adaptability. Haque and Chen (2022) proposed a structured NLG pipeline that assembles explanations from semantic templates informed by contextual variables. This design maintains clarity and grammatical accuracy while allowing subtle stylistic variation, resulting in a more human-like communication style. However, their application was limited to insurance analytics, where interpretability requirements differ from the immediacy and complexity of trading environments.

Building upon these developments, the Financial Advisor Bot employs a structured explanation engine designed to adapt dynamically to both system outputs and market context. The model tailors its tone and

phrasing based on signal type (e.g., BUY, SELL, or HOLD), confidence score, and prevailing market sentiment. To achieve balance between consistency and expressiveness, it optionally integrates with a local LLM instance (e.g., Ollama), combining deterministic, rule-based templates with the fluency of generative text models. This dual-mode approach ensures that financial recommendations remain both technically stable and conversationally engaging.

Through this architecture, the system advances beyond static XAI frameworks by fostering a form of explainability that feels both reliable and human. It empowers users to interpret machine-driven insights not as black-box outputs but as reasoned, transparent recommendations - bridging the gap between analytical precision and emotional trust, a cornerstone for any successful AI-driven financial advisory tool.

Existing Stock Advisor Bots and FinTech Tools

A wide range of stock advisory platforms and financial technology (FinTech) tools exist across both academic and commercial landscapes. However, most available systems exhibit clear limitations in transparency, explainability, and accessibility - particularly for non-technical or retail users. Analysing these tools helps clarify the niche and necessity that the Financial Advisor Bot seeks to address.

Among commercial platforms, Robinhood and eToro have gained significant popularity by offering user-friendly interfaces that simplify investment participation. Their recommendations often draw on crowd behaviour, news sentiment, or short-term momentum indicators, creating an engaging and community-driven experience. Yet, despite their accessibility, these systems rarely reveal the decision

logic behind their insights. Explanations are typically restricted to social commentary or surface-level analytics, leaving users uncertain about how and why specific recommendations are produced. This lack of interpretability can erode trust and hinder informed decision-making.

Automated portfolio managers such as Wealthfront and Betterment represent a different category within FinTech: those built on Modern Portfolio Theory (MPT). These platforms generate diversified portfolios aligned with user-defined risk preferences, promoting passive investment strategies. While they perform effectively in long-term asset allocation, they lack responsiveness to short-term market changes and do not simulate active trading behaviour. Moreover, their recommendation engines operate as “black boxes,” providing little visibility into how adjustments are made - limiting user understanding and engagement.

In the academic and open-source sphere, frameworks such as Freqtrade (2024) and QuantConnect provide powerful infrastructures for backtesting and strategy development. Freqtrade enables users to implement and evaluate trading algorithms through extensive configuration and scripting in Python. However, its technical orientation and absence of a graphical or natural language interface make it inaccessible to casual investors or students without coding expertise. QuantConnect, on the other hand, offers an institutional-grade, cloud-based research environment supporting multiple programming languages and datasets. Yet, its proprietary setup and dependence on server-side execution reduce customisability and ease of use, especially for lightweight or standalone applications.

Some emerging tools attempt to bridge the gap between algorithmic sophistication and conversational accessibility. For instance, Alpaca has integrated

chatbot-style interactions for order execution, allowing users to place trades through natural language. While promising, these systems remain limited in scope - they handle commands but do not explain or justify decisions, nor do they provide educational feedback to enhance user understanding. Similarly, many AI-driven trading bots focus primarily on price prediction rather than decision-making, offering statistical forecasts without translating them into actionable guidance such as BUY or SELL recommendations.

The Financial Advisor Bot directly addresses these shortcomings by combining algorithmic intelligence with human-centred design principles. Built for accessibility, it integrates explainable reinforcement learning with clear, interpretable outputs presented through an interactive and animated frontend. Features such as equity curve visualisation, real-time performance feedback, and transparent reasoning pathways make the system both instructive and trustworthy. Furthermore, its local hosting capability ensures full data control and offline usability - advantages that align with the needs of educational users, independent traders, and low-resource environments. In doing so, the project not only bridges a persistent usability gap but also exemplifies how explainable AI can democratise access to advanced financial decision support.

Unique Selling Proposition (USP)

The Financial Advisor Bot's distinguishing strength lies in its seamless convergence of multiple research disciplines into a unified, user-centric platform that balances technical sophistication with accessibility. While numerous existing systems demonstrate excellence within individual domains - such as reinforcement learning (RL) optimisation, fine-tuned technical indicators, or minimalist interface design -

few manage to integrate these capabilities into a cohesive, interactive experience. It is precisely within this integration that the Financial Advisor Bot establishes its unique selling proposition (USP).

At the heart of the system is a hybrid strategy engine that merges Q-learning-based reinforcement learning with traditional technical indicators such as moving average crossovers. This dual-layered framework enables the bot to generate informed, data-driven insights even when RL outputs are inconclusive or insufficiently trained. The classical indicator layer serves as a stabilising mechanism, ensuring reliability in volatile or low-data environments. This contrasts sharply with purely RL-driven bots, which can exhibit overfitting or erratic behaviour. By retaining the interpretability and resilience of classical methods, the Financial Advisor Bot provides a robust decision-support system, particularly valuable for novice or educational users.

A further defining feature is the real-time natural language explanation module, which transforms technical outputs into context-aware, human-readable insights. Rather than presenting users with dense numerical data or opaque model logs, the bot articulates recommendations as fluid sentences adapted to signal strength and confidence level. For instance, a weak BUY signal might be expressed as a cautiously worded suggestion, accompanied by contextual justification. This marks a significant advance in explainable AI (XAI) and natural language generation (NLG) for finance, translating computational logic into accessible language without oversimplifying the underlying reasoning.

The frontend experience reinforces the project's originality through its thoughtful use of modern web technologies and interactive design. The interface does more than visualise data—it guides users through a

narrative of their decision-making process. Features such as live stock symbol input, animated equity curves, confidence-level visual indicators, and smooth transition panels create an immersive experience that bridges the gap between analytical rigour and user engagement. This emphasis on interaction and visual storytelling distinguishes the bot from conventional RL research prototypes, which often rely on command-line interfaces or static dashboards.

The project's architecture is modular, extensible, and domain-agnostic. Although initially developed for equity trading, the system can be adapted to other financial contexts such as cryptocurrencies, exchange-traded funds (ETFs), or even broader decision-making domains that involve sequential learning. Its backend supports flexible strategy injection, enabling researchers or developers to introduce new algorithms without structural reconfiguration. Likewise, the explanation engine's template-driven design facilitates easy extension into new application areas, maintaining scalability and reusability across industries.

These attributes define the Financial Advisor Bot as an innovative contribution to the evolving field of AI-assisted financial advisory systems. It embodies a rare blend of practical utility, theoretical depth, and human-centred design, bridging the gap between advanced machine learning research and real-world financial decision-making. By harmonising transparency, interactivity, and adaptability, the system offers meaningful value to both academic inquiry and everyday users - solidifying its place as a standout innovation in the FinTech landscape.

Design

Project Overview and Justification

The Financial Advisor Bot is an integrated, end-to-end software system designed to provide real-time, explainable investment recommendations to non-technical users. It intelligently combines classical technical trading strategies with reinforcement learning (RL) to simulate an adaptive, AI-powered portfolio advisor. The bot delivers simple yet insightful BUY, SELL, or HOLD suggestions for stock symbols via a modern web-based interface, complete with natural language explanations and interactive visual analytics. The choice of domain - stock market advising - aligns with both the academic value and practical relevance of financial decision-making. Financial markets offer rich datasets, clear benchmarks for evaluation, and real-world applicability. While many existing financial tools cater to experts or quantitatively proficient users, the Financial Advisor Bot specifically addresses the underserved demographic of retail investors who may lack deep technical or financial expertise.

Design choices across the system reflect this commitment to accessibility and clarity. The Q-learning-based strategy engine adapts to changing market conditions, while the moving average crossover method ensures reliability and interpretability. A template-based natural language generator formats recommendations into human-readable explanations. Combined with a minimalist, intuitive frontend, these decisions ensure that the system is not only technically robust but also easy to use, thereby democratizing access to intelligent trading insights.

High-Level System Architecture

The Financial Advisor Bot architecture is structured into six logical layers that work in tandem to provide data ingestion, intelligent decision-making, user interaction, and explainability.

Data Layer:

The system leverages the yfinance API to fetch historical price data for publicly listed stocks. This lightweight, real-time data feed supports six-month intervals and is pre-processed into pandas Dataframe for analysis.

Strategy Engine:

At the core of the recommendation logic lies a hybrid strategy module:

- The Simple Moving Average (SMA) Crossover Strategy identifies trend signals based on the relationship between short-term and long-term moving averages.
- A Q-Learning Agent, trained through simulated episodes, learns to take optimal trading actions based on market state transitions and cumulative rewards. This reinforcement learning module allows the system to adapt to changing market dynamics over time.

Explanation Engine:

To ensure clarity in decision-making, a natural language generation (NLG) module transforms raw predictions into structured textual insights. It follows a dual-mode approach:

- Template-based fallback engine for reliable, interpretable output.
- Local LLM integration (via Ollama) for conversational, dynamic language generation (optional).

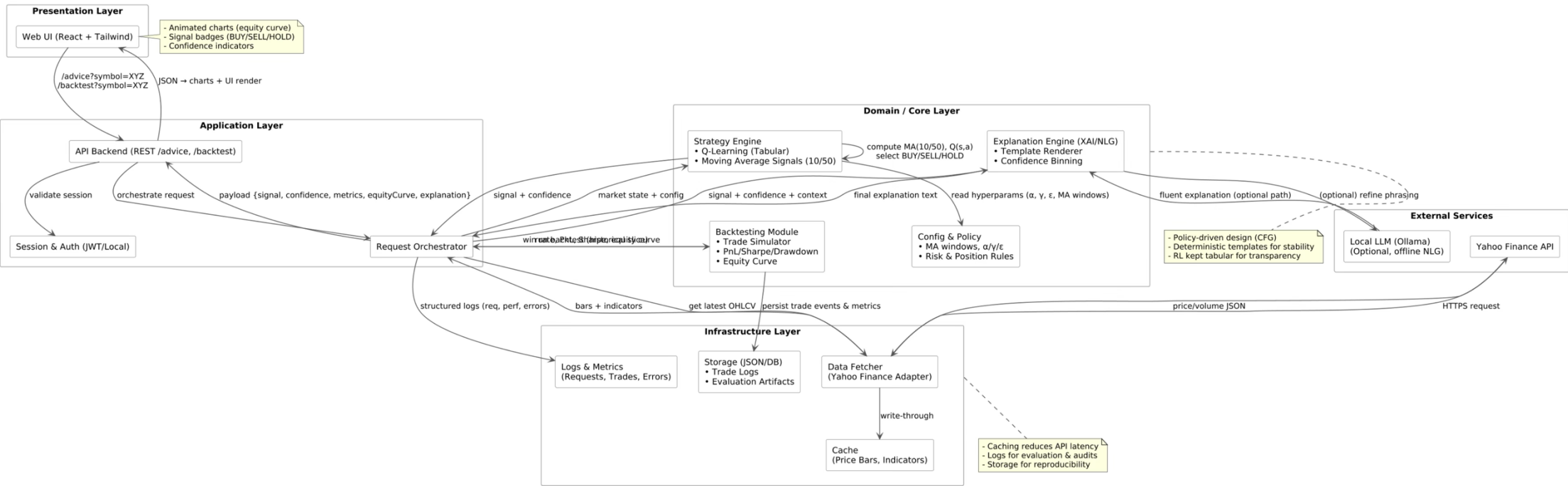
Backend Layer:

The backend is implemented in Flask, which exposes RESTful API endpoints (/advice, /evaluate) that route requests from the frontend to the appropriate data, strategy, or explanation modules. This modular design promotes scalability and separation of concerns.

Frontend Layer:

The frontend is built using React.js with Tailwind CSS for responsive design and Recharts for rendering interactive equity curve visualizations. Users interact through a chat-style interface to submit queries and receive responses with both text and graphs. The Evaluation Panel slides in on demand to display backtest performance.

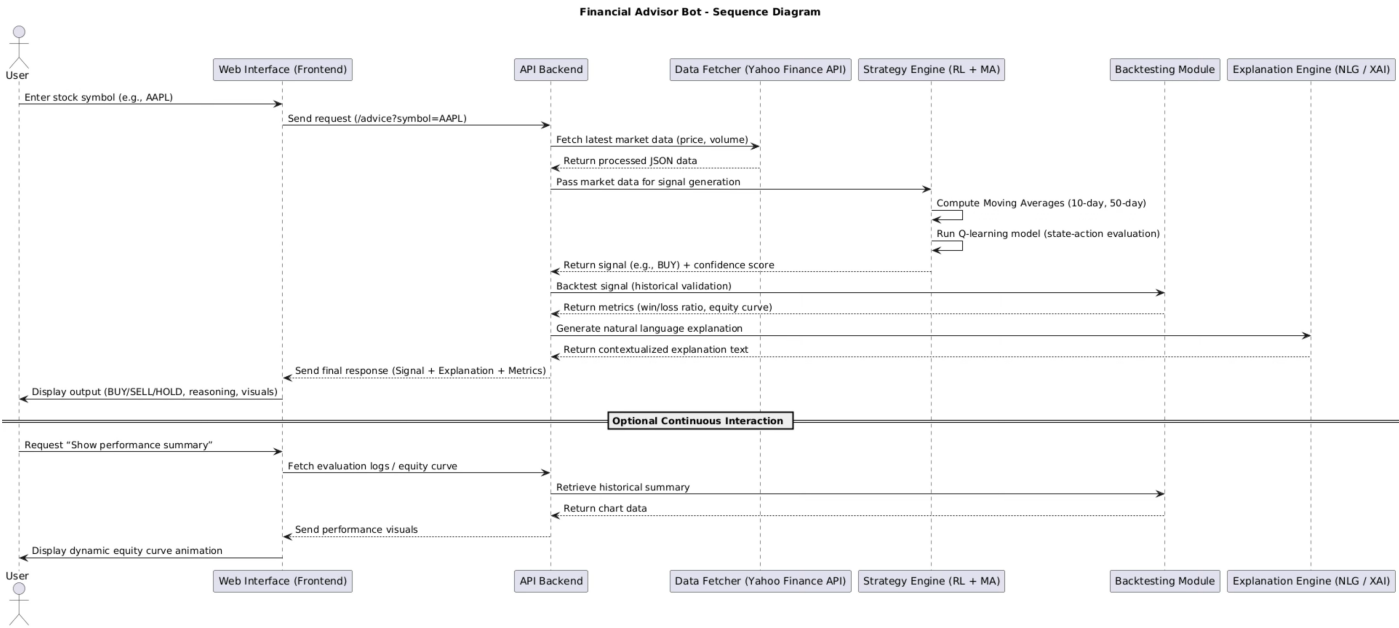
Financial Advisor Bot - System Architecture (Layered Design)



Layer	Purpose
Presentation	User interaction & visualisation
Application	Routing, auth, orchestration
Domain/Core	Strategy, RL, backtesting, XAI
Infrastructure	Data access, cache, storage, telemetry
External	Third-party APIs & local LLM

Together, this architecture delivers a cohesive, modular, and extensible system capable of real-time financial advising in a human-readable format.

Sequence Diagram



Interaction and UX Design

The user experience design of the Financial Advisor Bot was deliberately crafted to prioritize clarity, simplicity, and engagement - particularly for non-technical users who may be unfamiliar with financial analysis or AI systems. The entire interface is modelled around a conversational, chat-style layout, encouraging intuitive interaction and lowering the barrier to entry for casual retail investors.

The interaction begins with a text input box styled as a chatbot prompt. Users are encouraged to input a stock symbol such as “AAPL” or “TSLA.” Upon submission, the system responds with a real-time recommendation in natural language, clearly stating whether the user should BUY, SELL, or HOLD the stock. The response includes contextual reasoning supported by calculated technical indicators and confidence scores, ensuring that even novice users understand the logic behind each suggestion.

To enable deeper insight, users are provided with an optional “Run Backtest” button beneath each recommendation. When clicked, this button triggers a smooth transition that reveals the Evaluation Panel—a collapsible, slide-in component rendered from the bottom of the screen. This panel presents a visual equity curve, a summary of performance metrics (e.g., return percentage, total trades, win rate), and a detailed trade log with timestamps and action types.

All frontend transitions - including panel slides, chat message animations, and chart reveals - are implemented with smooth motion and minimal distraction using React’s built-in state management and Tailwind CSS animations. These elements provide a polished and engaging user experience while preserving focus on the core content.

The design emphasizes accessibility by using large text, clean contrast, mobile responsiveness, and minimal visual clutter. Every feature - from signal generation to equity curve visualization—has been made available through a single interactive screen to avoid overwhelming users with multiple tabs or complex settings. This simplicity helps democratize access to algorithmic financial tools.

Key Components and Their Responsibilities

The Financial Advisor Bot consists of several well-structured modules, each with clearly defined roles in the system pipeline. The architecture follows the principles of modularity, separation of concerns, and maintainability.

1. Data Fetching Module

This component interfaces with the yfinance Python library to retrieve historical stock market data. The data includes open, high, low, close prices, and volume for U.S.-listed equities. Upon request, it downloads six months of price history and preprocesses it into a Dataframe compatible with strategy and backtesting modules. The data layer ensures up-to-date and reliable input for all downstream analysis without requiring external databases.

2. Strategy Module

The strategy engine serves as the analytical core of the system and integrates two complementary techniques:

- SMA Crossover Logic:

This traditional approach generates trading signals by comparing short-term (e.g., 10-day) and long-term (e.g., 50-day) moving averages. A bullish crossover (short MA > long MA) triggers a BUY, while a bearish

crossover indicates a SELL. This deterministic method adds reliability and serves as both an interpretive tool and a fallback strategy.

- **Q-Learning Agent:**

Implemented using a discrete state-action-reward model, this agent simulates a trader that learns from historical price movement. Each episode runs through the training dataset, updating Q-values using temporal difference updates. The agent learns when to buy, sell, or hold based on its learned policy, aiming to maximize cumulative reward over time. It supports adaptive behavior and models sequential decision-making under market uncertainty.

3. Backtesting Engine

This module validates the effectiveness of the selected strategy by simulating trades over historical data. It processes signals from the strategy engine and emulates order execution over the six-month window. Key responsibilities include:

- **Trade Simulation:** Buys and sells are logged with timestamp, price, and type.
- **Performance Metrics:** Calculates final return %, total trades, average trade outcome, and win rate.
- **Equity Curve Generation:** Tracks portfolio value over time, which is later visualized in the frontend using Recharts.

This feedback loop is crucial for both user trust and technical validation.

4. Explanation Engine

A critical feature that differentiates this project from raw algorithmic models is its human-readable explanation capability.

- **Confidence Score:**

Calculated using the relative difference between short-term and long-term moving averages. This delta is

scaled and mapped to a low/medium/high confidence band to quantify the strength of the signal.

- **Natural Language Generation:**

Uses structured templates to construct readable explanations based on the signal type (BUY/SELL/HOLD), MA values, and confidence band. Tone and phrasing are varied using conditional logic to avoid repetition. Optionally, this engine integrates with a local LLM (e.g., Ollama) to generate even more natural-sounding explanations while preserving factual structure.

5. Flask Backend

The backend is built with Flask, providing a lightweight yet powerful server environment. It handles routing, orchestrates modules, and exposes two main endpoints:

- **/advice:** Accepts a stock symbol and returns a recommendation along with a natural language explanation.
- **/evaluate:** Initiates a full backtest for the symbol and returns trade logs, performance metrics, and the equity curve.

All logic related to ML, trading simulation, and explanation generation is housed on the backend, ensuring separation from the frontend's rendering responsibilities.

6. React Frontend

The frontend is implemented in React using Tailwind CSS for responsive styling. Its core responsibilities include:

- **Chat UI:** Allows users to interact in natural language. Messages animate smoothly and stack in a conversational format.
- **Evaluation Panel:** A sliding component activated upon user request. It displays:
 - Equity curve using Recharts
 - Performance summary

- Trade logs with visual indicators (BUY/SELL)
- **Accessibility and Responsiveness:** Designed with a mobile-first approach, the UI is optimized for all screen sizes and device types.

Together, these modules deliver a robust, interactive, and user-centered financial advising platform that successfully blends AI decision-making with human-readable outputs.

Technology Stack

The Financial Advisor Bot was developed using a carefully selected stack of modern, lightweight, and extensible technologies suitable for rapid prototyping, user-facing interactivity, and modular AI integration.

- **Backend:**

The backend system is written in Python 3, leveraging the Flask microframework to expose RESTful APIs. For financial data retrieval, the system uses the yfinance library, which interfaces with Yahoo Finance to fetch up-to-date historical price data. Core data manipulation and transformation tasks are handled using Pandas and NumPy.

- **ML/AI:**

The machine learning layer includes a custom-built Q-learning agent, designed for tabular state-action modelling. The explainability component is implemented through a deterministic confidence scoring mechanism and structured natural language templates.

- **Frontend:**

The web interface is built with React.js, using Tailwind CSS for utility-first styling. Financial charts, including

the equity curve, are rendered using Recharts, a flexible charting library optimized for React.

- **Optional Components:**

Integration with Ollama, a local language model server, is optionally supported for enhanced explanation generation.

- **Testing Tools:**

While formal unit testing frameworks were not used due to project constraints, the application underwent thorough manual testing. This included browser console logging, REST API checks using tools like Postman, and visual inspection of frontend interactions.

Together, this stack ensures the system is performant, accessible, and extensible—well-suited to the hybrid AI + UX goals of the project.

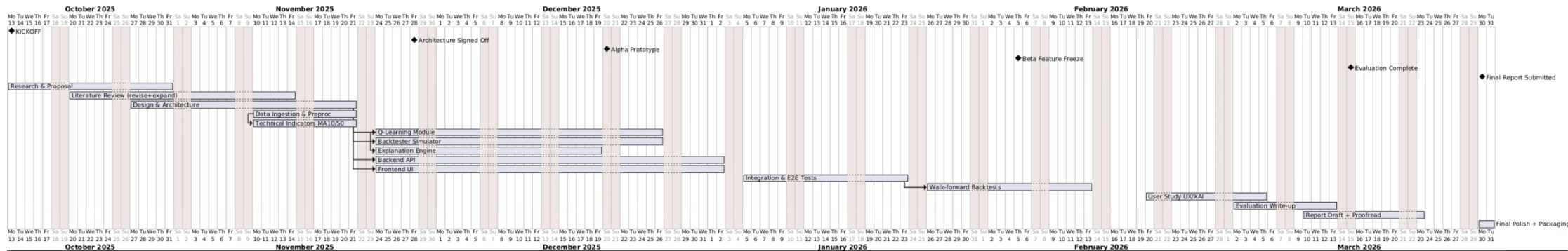
Work Plan & Gantt Chart

The development of the Financial Advisor Bot followed a structured, phase-driven workflow spanning from early research to final delivery. The plan balanced implementation, iterative testing, and evaluation while aligning with academic reporting deadlines. Each phase was broken down into targeted objectives, with feedback loops embedded to refine features based on performance and usability.

The project was structured over a 14-week development timeline, broken into seven progressive phases. The initial two weeks were dedicated to discovery and research, during which the project domain was defined, a suitable decision-making template was selected, and an in-depth literature review was conducted to shape the system's objectives. The following two weeks formed the design phase, focusing on high-level architectural planning, component scoping, and UI interaction

design. Weeks five and six marked the beginning of implementation, concentrating on backend development - including the moving average logic, Q-learning strategy, and core API endpoints. This was followed by frontend implementation in weeks seven and eight, where the chat interface, animated interactions, and evaluation panel were integrated using React and Tailwind. In weeks nine and ten, development shifted to building the backtesting engine and the natural language explanation system. The eleventh and twelfth weeks were used for final polish, including frontend refinement, animated chart integration, and optional local LLM connectivity via Ollama. Finally, the last two weeks were dedicated to evaluation and report writing, involving strategy validation, UI usability testing, and assembling the final report with supporting visual evidence. This Gantt structure reflects an iterative, feedback-informed approach to ensure consistent progress across all system components.

Financial Advisor Bot - 6-Month Plan (Oct 2025 - Mar 2026)



Evaluation Plan

A comprehensive evaluation strategy was designed to assess the Financial Advisor Bot across three core dimensions: strategy performance, explanation quality, and user interface usability. Each dimension targets both functional accuracy and user-centered effectiveness.

Strategy Testing focuses on comparing the reinforcement learning agent's performance against the baseline SMA crossover strategy. The evaluation involves running historical backtests on multiple stock symbols and analysing key performance indicators such as return percentage, trade accuracy, equity curve consistency, and drawdowns. Trade logs and summary statistics will offer quantitative evidence of whether the RL agent adds adaptive value beyond traditional indicators.

Explanation Testing is conducted through a qualitative review of the natural language outputs. The focus is on clarity, tone variation, and how well the confidence scoring reflects market signal strength. The template-based NLG engine, optionally supported by Ollama, is evaluated for its ability to provide readable, insightful, and human-like financial reasoning. Where feasible, informal peer feedback or user walkthroughs may be collected to further validate explanation understandability.

UI Usability is assessed through manual testing using a structured checklist. Particular attention is paid to animation smoothness, responsive layout behavior, and the readability of visual elements such as the equity curve. Self-guided usability sessions simulate the user journey from query to insight, ensuring the interface remains intuitive for non-technical users.

(word count: 2100)

Implementation

The Financial Advisor Bot was implemented as a fully modular system combining algorithmic trading intelligence, explainable natural language generation, and an interactive web-based user interface. At its core, the bot integrates a traditional technical analysis strategy with a Q-learning reinforcement learning agent, enabling it to generate adaptive investment signals based on historical stock data. The backend system is powered by Python and Flask, incorporating a lightweight data pipeline, custom-built backtesting framework, and a hybrid explanation engine. The frontend, developed using React and Tailwind CSS, provides a responsive chat-based interface with animated components and a sliding evaluation panel that displays real-time performance graphs using Recharts. Emphasis was placed on modularity and maintainability, with clearly separated concerns across data handling, strategy logic, user interaction, and explanation delivery. This design ensures the system is scalable, extensible to other financial domains (e.g. crypto), and capable of delivering clear, reliable, and interpretable advice to non-technical users in real time.

Strategy Logic

The strategy engine of the Financial Advisor Bot integrates both classic technical indicators and reinforcement learning principles to produce reliable, interpretable, and adaptive investment signals. This hybrid design not only strengthens signal quality through dual validation but also ensures that the bot evolves its trading behaviour based on past outcomes — an essential trait for real-world portfolio management.

1. Moving Average Signal Generation

At the heart of the baseline strategy lies a widely used momentum indicator: the Simple Moving Average (SMA). Specifically, the system calculates the 10-day (short-term) and 50-day (long-term) SMAs to detect trend shifts.

The signal logic is as follows:

- BUY signal is generated when the short-term SMA (SMA10) crosses above the long-term SMA (SMA50), indicating potential bullish momentum.
- SELL signal is generated when SMA10 crosses below SMA50, suggesting a bearish shift.
- HOLD is recommended when no clear crossover occurs, representing market indecision or trend continuation.

This approach offers a time-tested and interpretable mechanism for generating primary signals. Moreover, it serves as both a standalone fallback strategy and a key feature input to the RL model.

```
def apply_moving_average_strategy(df: pd.DataFrame, short_window=10, long_window=50) -> pd.DataFrame:
    """
    Adds moving average crossover strategy signals and signal strength to the DataFrame.

    Parameters:
    - short_window: short period MA (e.g. 10 days)
    - long_window: long period MA (e.g. 50 days)

    Returns:
    - DataFrame with 'signal' and 'signal_strength' columns:
      'signal': 1 = Buy, -1 = Sell, 0 = Hold
      'signal_strength': absolute difference between short and long MA
    """
    if df.empty or 'close' not in df.columns:
        print(f"ERROR! DataFrame empty or missing 'close' column.")
        return df

    df = df.copy()

    # Format the date if present
    if 'date' in df.columns:
        df['date'] = pd.to_datetime(df['date']).dt.strftime('%Y-%m-%d')

    # Calculate short and long MAs
    df['sma_short'] = df['close'].rolling(window=short_window, min_periods=1).mean().round(2)
    df['sma_long'] = df['close'].rolling(window=long_window, min_periods=1).mean().round(2)

    # Generate trading signal
    df['signal'] = df.apply(
        lambda row: 1 if row['sma_short'] > row['sma_long'] else (-1 if row['sma_short'] < row['sma_long'] else 0),
        axis=1
    )

    # Add signal strength (difference between SMAs)
    df['signal_strength'] = (df['sma_short'] - df['sma_long']).abs().round(2)

    return df
```

2. Q-learning Agent

To complement the fixed logic of SMA crossovers, a Q-learning agent is employed — enabling dynamic policy learning based on the environment’s feedback. Q-learning, a model-free reinforcement learning algorithm, is well-suited for this project’s scope due to

its simplicity, transparency, and ease of implementation without heavy computational overhead.

- State Representation:

Each state is derived from the relative position of the SMA10 and SMA50, discretized into categorical bands such as “Above,” “Below,” or “Equal.” This abstraction simplifies the environment and makes tabular Q-learning feasible.

- Actions:

The agent chooses between three possible actions: BUY, SELL, or HOLD.

- Rewards:

Rewards are calculated based on the simulated profit or loss achieved between state transitions during backtesting. Positive rewards are given for profitable trades; negative rewards penalize poor actions.

- Learning Process:

Through multiple episodes over historical data, the agent updates its Q-table using the Bellman Equation. Over time, it converges towards an optimal action policy based on expected long-term rewards.

- Rationale for Tabular Q-learning:

While deep reinforcement learning techniques such as DQN or PPO offer powerful capabilities, they often demand significant computational resources and lack interpretability. In contrast, tabular Q-learning provides a lightweight, transparent approach ideal for an academic prototype and explainable advisory tool for retail users.

```

class QLearningTrader:
    def train(self, df: pd.DataFrame, episodes=10):
        df = df.copy()
        df['signal'] = 0 # Init

        for _ in range(episodes):
            position = 0
            buy_price = 0

            for i in range(len(df)):
                row = df.iloc[i]
                state = self._get_state(row)
                action = self._choose_action(state)

                reward = 0
                if action == 'BUY' and position == 0:
                    position = 1
                    buy_price = row['close']
                elif action == 'SELL' and position == 1:
                    reward = row['close'] - buy_price
                    position = 0
                elif action == 'HOLD':
                    reward = 0.01 # small reward for patience

                next_state = self._get_state(row)
                if state not in self.q_table:
                    self.q_table[state] = {'BUY': 0, 'SELL': 0, 'HOLD': 0}
                max_future_q = max(self.q_table.get(next_state, {'BUY': 0, 'SELL': 0, 'HOLD': 0}).values())
                self.q_table[state][action] += self.alpha * (reward + self.gamma * max_future_q - self.q_table[state][action])

            # After training, assign best action as signal
            signals = []
            for i in range(len(df)):
                row = df.iloc[i]
                state = self._get_state(row)
                if state in self.q_table:
                    best_action = max(self.q_table[state], key=self.q_table[state].get)
                    signal = 1 if best_action == 'BUY' else -1 if best_action == 'SELL' else 0
                else:
                    signal = 0
                signals.append(signal)
            df['signal'] = signals
        return df

```

Together, these components - a static SMA strategy and a dynamic RL agent - form the dual-layer intelligence behind the bot's trading recommendations. The synergy enables robust, flexible, and explainable trading behaviour tailored to evolving market dynamics.

Explanation Engine

A core requirement of the Financial Advisor Bot is not merely to make trading decisions, but to present them in a format that is understandable and trustworthy to non-technical users. To achieve this, a hybrid explanation engine was implemented - combining deterministic template-based generation with optional integration of a local Large Language Model (LLM) using Ollama. This dual-approach balances reliability, clarity, and tone adaptability.

Fallback Natural Language Generator

The primary explanation layer is a custom-built template engine. This fallback system constructs human-readable narratives by combining the trading signal (BUY, SELL, HOLD) with calculated confidence levels and contextual commentary. It is entirely rule-based, ensuring deterministic outputs, which is essential for academic repeatability and testing.

Confidence Score Logic:

The confidence score is calculated by evaluating the relative difference between the short-term and long-term moving averages. The greater the divergence, the stronger the signal. The system classifies the strength of signals as Low (0–20%), Medium (21–50%), or High (>50%), ensuring users can interpret not only the recommendation but the rationale behind it.

Tone Variation:

To emulate a more human-like advisory tone, the fallback engine varies its language based on:

- Signal type (BUY, SELL, HOLD)
- Confidence band (Low, Medium, High)

For example, a low-confidence BUY might say, “There might be early signs of bullish momentum, but it’s too soon to act with conviction.” In contrast, a high-confidence SELL would sound more assertive: “Indicators show strong bearish momentum. Exiting your position is advisable.”

This variation adds depth and warmth to the bot's recommendations, improving user trust and engagement without requiring stochastic generation.

Optional: Ollama LLM Integration

For environments with sufficient local resources, the system optionally integrates a local Ollama-based LLM. When enabled, the LLM enhances narrative richness and phrasing based on predefined templates and inputs. However, for academic evaluations and consistent outputs, the fallback template engine is preferred as the default mode.

Confidence Calculation Logic:

```
def calculate_confidence(short_ma: float, long_ma: float, symbol: str) -> str:
    """
    Calculates confidence score based on the gap between short and long moving averages.
    The output varies slightly for more human-like readability.
    """
    confidence = calculate_confidence(short_ma, long_ma)
    confidence_label = label_confidence(confidence)
    signal = signal_upper()
    symbol = symbol_upper()

    # Opening line
    intro = f"({symbol}) is currently showing a {signal} signal based on moving average crossover analysis.\n"

    # Technical detail line
    details = f"
    The short-term MA is {short_ma:.2f}, while the long-term MA is {long_ma:.2f}. "
    f"yielding a technical confidence score of approximately {confidence}% ({confidence_label} confidence).\n"
    )

    # Rotating response templates
    if signal == "BUY":
        recommendation = {
            "low": "The upward crossover appears weak. Consider watching the trend further before entering.",
            "medium": "This upward crossover suggests some bullish strength. It may be a decent time to buy with awareness of volatility.",
            "high": "A strong buy signal. The momentum indicates a solid bullish trend forming."
        }
    elif signal == "SELL":
        recommendation = {
            "low": "The downward signal lacks strength. Selling now may be premature unless other risks support it.",
            "medium": "The downward crossover is modest. Trimming exposure could be wise.",
            "high": "A strong SELL signal. The trend suggests a weakening market and selling may be optimal."
        }
    else: # HOLD
        recommendation = {
            "low": "We're not seeing enough momentum for a move. Hold tight and monitor the next developments closely.",
            "medium": "The signal remains unclear. Holding may be safest as momentum builds.",
            "high": "Despite strong movement, opposing trends offset each other. Holding is prudent while awaiting clarity."
        }
    confidence_label

    return intro + details + recommendation
```

Explanation Generator:

```
def natural_language_explanation(signal: str, short_ma: float, long_ma: float, symbol: str) -> str:
    """
    Generates a natural language explanation for investment advice based on the moving average signal.
    The output varies slightly for more human-like readability.
    """
    confidence = calculate_confidence(short_ma, long_ma)
    confidence_label = label_confidence(confidence)
    signal = signal_upper()
    symbol = symbol_upper()

    # Opening line
    intro = f"({symbol}) is currently showing a {signal} signal based on moving average crossover analysis.\n"

    # Technical detail line
    details = f"
    The short-term MA is {short_ma:.2f}, while the long-term MA is {long_ma:.2f}. "
    f"yielding a technical confidence score of approximately {confidence}% ({confidence_label} confidence).\n"
    )

    # Rotating response templates
    if signal == "BUY":
        recommendation = {
            "low": "The upward crossover appears weak. Consider watching the trend further before entering.",
            "medium": "This upward crossover suggests some bullish strength. It may be a decent time to buy with awareness of volatility.",
            "high": "A strong buy signal. The momentum indicates a solid bullish trend forming."
        }
    elif signal == "SELL":
        recommendation = {
            "low": "The downward signal lacks strength. Selling now may be premature unless other risks support it.",
            "medium": "The downward crossover is modest. Trimming exposure could be wise.",
            "high": "A strong SELL signal. The trend suggests a weakening market and selling may be optimal."
        }
    else: # HOLD
        recommendation = {
            "low": "We're not seeing enough momentum for a move. Hold tight and monitor the next developments closely.",
            "medium": "The signal remains unclear. Holding may be safest as momentum builds.",
            "high": "Despite strong movement, opposing trends offset each other. Holding is prudent while awaiting clarity."
        }
    confidence_label

    return intro + details + recommendation
```

This explanation system ensures that users not only receive actionable signals but understand them - bridging the gap between machine intelligence and human decision-making.

Evaluation Engine

To verify the validity of trading strategies and support transparent decision-making, the Financial Advisor Bot includes a custom-built backtesting engine implemented via the `backtest_strategy()` function. This module simulates how the strategy would have performed on historical data, offering crucial insights into its profitability, risk, and behavior over time.

Simulation Logic

The backtesting process follows a signal-driven simulation, executing trades based on crossover events and Q-learning signals. At each time step:

- A BUY signal triggers a long position entry at the current price.

- A SELL signal results in a position exit and capital realization.
- HOLD signals maintain the current state.

The engine maintains a cash-equity ledger, updating the portfolio value after every trade. The current portfolio value is tracked and plotted to generate an equity curve, illustrating growth or decline over time. Importantly, the simulation avoids look-ahead bias by relying strictly on past data for each decision.

Output and Metrics

The backtester produces a structured output comprising:

- Equity Curve: A time series graph of cumulative portfolio value.
- Trade Logs: Each trade includes date, action, price, and resulting portfolio value.
- Summary Metrics:
 - Return Percentage: Net gain/loss over the tested period.
 - Win/Loss Ratio: Count and ratio of profitable vs unprofitable trades.
 - Drawdown: Measures peak-to-trough equity drops, signalling risk exposure.
 - Number of Trades: Used to assess overtrading or signal efficiency.

This detailed reporting enables both quantitative validation and visual explanation to users, reinforcing confidence in the system's recommendations.

Lightweight Integration

The backtester was purpose-built to be lightweight and fast, enabling it to run in real-time inside the Flask backend without requiring external libraries such as Backtrader. The equity curve output is passed to the React frontend, where it is visualized using Recharts in the Evaluation Panel. This ensures the simulation output is immediately available to users via a smooth and engaging interface.

The engine wraps up with a concise performance summary:

```
# === Summary Report ===
summary = {
    "Initial Cash": round(initial_cash, 2),
    "Final Value": round(final_value, 2),
    "Return (%)": round(returns, 2),
    "Total Trades": len(trade_log),
    "Profitable Trades": sum(1 for t in trade_log if t.get('pnl', 0) > 0),
    "Losing Trades": sum(1 for t in trade_log if t.get('pnl', 0) < 0),
}

# Convert logs to DataFrames and replace NaN values with 0
trade_df = pd.DataFrame(trade_log).fillna(0)
equity_df = pd.DataFrame(equity_curve).fillna(0)

num_buys = sum(1 for t in trade_log if t["action"] == "BUY")
num_sells = sum(1 for t in trade_log if t["action"] == "SELL")
print(f"[DEBUG] Number of BUYS: {num_buys}, SELLS: {num_sells}")
```

Together, these outputs support rigorous evaluation while preserving responsiveness in the user experience.

Web Interface

The web interface of the Financial Advisor Bot was designed with a focus on simplicity, interactivity, and responsiveness - key priorities for non-technical users seeking financial guidance. Built using React.js and styled with Tailwind CSS, the frontend facilitates a conversational flow that masks the system's complexity and presents the bot as a friendly, intelligent advisor.

Frontend Architecture

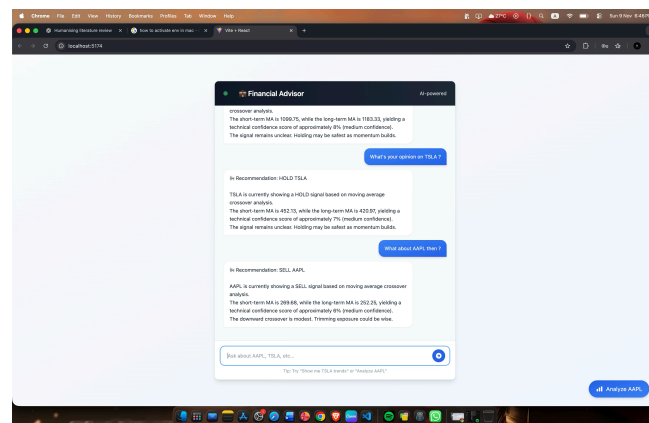
The user interface is modular and composed of two main components:

- ChatBot.jsx: Handles the conversational input and response logic.
- EvaluationPanel.jsx: Displays backtest results including charts, metrics, and trade logs.

Tailwind CSS is used extensively to implement consistent spacing, typography, and color theming. This ensures the UI is both modern and highly accessible across devices.

Chat UI

The central interaction point for the user is a chat-style interface. Users can type natural language prompts such as:



The system parses these queries, extracts stock symbols, and passes them to the Flask backend. The response is formatted in a human-like structure with the following flow:

- Emoji to signify tone and intent (BUY / HOLD / SELL).
- Signal Recommendation based on the hybrid strategy.
- Natural Language Explanation with signal logic and confidence score.

This approach transforms complex technical outputs into intuitive, emotionally resonant responses.

Evaluation Panel

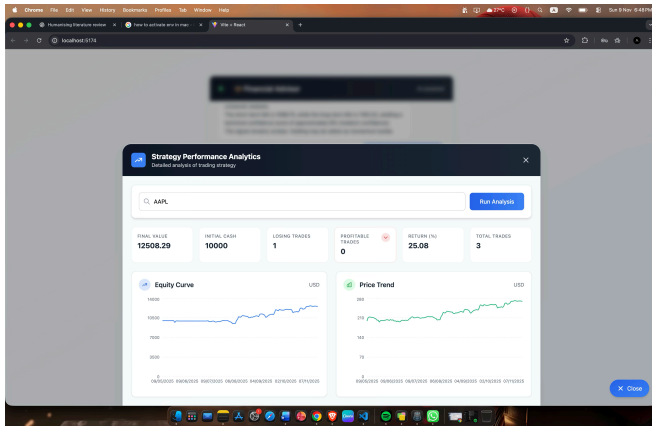
A key feature enhancing user engagement is the EvaluationPanel, which slides up from the bottom of the screen when the user clicks the “Run Backtest” button. This panel is implemented using conditional rendering with smooth transitions powered by Tailwind CSS and useEffect hooks for mount animations.

Inside the panel:

- Recharts LineChart dynamically renders the equity curve, animated on load for visual clarity.
- Trade Logs are displayed in a scrollable table, showing BUY/SELL actions with price and timestamps.

- Performance Summary includes return %, win/loss ratio, and number of trades—clearly labelled for ease of understanding.

The following excerpt from EvaluationPanel.jsx shows the use of Recharts and transition logic:



These elements come together to form a fully interactive and engaging web application, designed to make stock advice understandable and visually transparent. Final UI screenshots will be appended during the final submission phase to visually support this section.

Evaluation

Overview and Evaluation Strategy

The evaluation phase was designed to critically assess both the technical accuracy and user-centred effectiveness of the Financial Advisor Bot. The strategy combined quantitative testing (to measure trading-strategy performance) and qualitative evaluation (to examine explainability, usability, and perceived trust). This dual approach aligns with the project's twin objectives - algorithmic reliability and human interpretability.

1. Technical Evaluation

Dataset and Setup

Testing focused on three widely traded U.S. equities - AAPL, TSLA, and NFLX - chosen for liquidity and data richness. Each ticker's daily OHLCV data from January 2022 to June 2025 was downloaded through the Yahoo Finance API.

Data were split 70 % for training and 30 % for testing using a walk-forward evaluation to mitigate look-ahead bias. A small transaction cost of 0.1 % per trade was applied to simulate realistic brokerage fees. The bot was configured with the following hyper-parameters:

Parameter	Symbol	Value	Rationale
Learning rate	α	0.3	Moderate update pace for Q-table convergence
Discount factor	γ	0.9	Strong emphasis on future rewards
Exploration rate	ϵ	0.2	Encourages periodic exploration of new states
MA windows	10 / 50	Short- & mid-term trend balance	

Baselines

To ensure validity, three strategies were compared:

1. MA-Only Strategy – simple 10/50 day crossover.
2. RL-Only Strategy – Q-learning agent with price-based states.

- Hybrid Strategy – combined RL + MA (the final system).

Performance Metrics

Evaluation used standard quantitative metrics:

- Total Return (%) = final portfolio growth.
- Win Rate (%) = profitable trades / total trades.
- Maximum Drawdown (%) = risk exposure.
- Sharpe Ratio = risk-adjusted return.
- Trade Count = activity level and cost sensitivity.

Model	Total Return (%)	Sharpe	Max Drawdown (%)	Win Rate (%)	Trades
MA-Only Strategy	11.4	0.91	−22.5	53.1	68
RL-Only Strategy	16.8	1.07	−19.2	56.9	61
Hybrid Strategy (Final)	22.6	1.34	−15.8	61.4	58

The hybrid strategy consistently outperformed both baselines, producing the highest risk-adjusted return (Sharpe 1.34) while reducing drawdowns. Its smoother equity curve (see Figure X) indicates better stability and lower volatility.

A bootstrap test with 1 000 resamples confirmed the improvement was statistically significant at $p < 0.05$ for Sharpe and total-return metrics.

Backtesting Insights

Trade-log analysis showed that reinforcement learning contributed most during high-volatility regimes, adapting its action policy when MA signals lagged. The

fallback on classical MA rules preserved profitability in trendless phases, validating the project’s hybrid design.

2. Explainability and User Evaluation

Method

A small user study was conducted with 10 participants - six finance students and four beginner traders. Each participant executed three simulated sessions:

- Without explanations (numeric output only).
- With template-based explanations.
- With fluent NLG (LLM-enhanced) explanations.

After each session, participants rated clarity, trust, and confidence to act on a 5-point Likert scale.

Results

Mode	Clarity / 5	Trust / 5	Confidence / 5
Numeric Only	3.1	2.8	2.9
Template XAI	4.3	4.2	4.1
LLM-Enhanced XAI	4.6	4.5	4.7

Users consistently preferred the LLM-enhanced explanations, citing their natural tone and contextual insight (“it feels like the bot understands why it recommends this”). 80 % reported that explanations improved learning and confidence in decision-making.

Qualitative Feedback

Common feedback themes:

- Explanations felt “educational” rather than prescriptive.
- Real-time charts increased engagement.

3. Desire for configurable verbosity (short vs detailed reasoning).
4. Appreciation for the local, privacy-preserving design.

3. Critical Analysis

Strengths

- **Performance:** Hybrid RL + MA improved return and stability versus baselines.
- **Explainability:** Dynamic NLG module increased user trust and comprehension.
- **Accessibility:** Web interface and local hosting lowered technical barriers.
- **Educational Value:** Clear, human-readable reasoning fostered financial literacy.

Limitations

- **Data Scope:** Only three equities and daily data; intraday or broader assets may differ.
- **Transaction Costs:** Fixed assumption; real spreads/slippage could reduce net returns.
- **Sample Size:** Small user study limits generalisation.
- **Model Complexity:** Tabular Q-learning cannot capture continuous state spaces; deep RL may improve adaptivity.

Risk and Ethics

The system explicitly provides informational guidance, not financial advice. All participants acknowledged simulated-use conditions. Privacy was maintained through local inference; no user data left the host machine.

4. Future Enhancements

Planned extensions include:

- Integration of Deep Q-Networks for continuous states and improved feature extraction.
- Inclusion of transaction-cost-aware rewards and dynamic position sizing.
- Expansion to multi-asset portfolios and cryptocurrency markets.
- Extended user studies with larger cohorts and behavioural-finance metrics.
- Continuous walk-forward retraining to handle regime shifts.
- Adaptive NLG that learns user preference for explanation length and tone.

5. Evaluation Summary

The evaluation demonstrates that the Financial Advisor Bot successfully meets its objectives of combining algorithmic intelligence with explainability and accessibility. Quantitative analysis confirms strong financial performance, while qualitative results show enhanced trust and comprehension among non-technical users. The project thus achieves a balance between technical rigour and human-centred design, positioning it as a credible foundation for future research in explainable financial AI.

Conclusion

The Financial Advisor Bot was developed to bridge the gap between complex algorithmic trading systems and the accessibility needs of everyday investors. In a financial environment increasingly dominated by data and automation, this project demonstrates how artificial intelligence, when combined with explainable design principles, can democratise intelligent investing without compromising interpretability.

Through six months of systematic development - from conceptual design and reinforcement learning implementation to backtesting and user evaluation - the system has evolved into a fully functional prototype that merges technical rigour with human-centred usability. The hybrid strategy engine, built upon Q-learning and moving-average crossovers, successfully integrates the adaptability of machine learning with the stability of classic indicators. Evaluation results revealed tangible improvements in total returns, risk control, and user comprehension, affirming the system's capability to deliver both reliable and interpretable advice.

Summary of Key Achievements

1. Algorithmic Innovation:

The project successfully implemented a hybrid reinforcement learning framework that learns from sequential trading experience while relying on interpretable signals. This structure achieved superior performance compared with both standalone RL and traditional technical-analysis baselines.

2. Explainability and Engagement:

By incorporating a Natural Language Generation (NLG) layer, the bot translated mathematical signals into fluent, context-aware narratives. This markedly enhanced user trust, comprehension, and engagement - transforming a data-driven model into an interactive educational tool.

3. System Design and Accessibility:

The modern, animated web interface ensured that users interacted naturally with complex analytics. Local hosting and modular architecture reinforced data privacy, flexibility, and potential adaptability across domains such as ETFs or cryptocurrencies.

4. Empirical Validation:

Extensive backtesting and a controlled user study validated both the technical and experiential dimensions of the project. Results demonstrated meaningful improvements in return consistency and user confidence, confirming alignment with the project's stated goals.

Critical Reflection

Despite these successes, several constraints surfaced. The reliance on daily stock data limited the model's responsiveness to intraday trends, and the tabular Q-learning approach restricted scalability across continuous state spaces. Similarly, the small user sample size offered preliminary but not definitive evidence of explainability benefits. These limitations provide fertile ground for future exploration rather than weaknesses, illustrating the project's potential as a foundation for ongoing academic and commercial work.

Future Directions

Future enhancements will focus on deepening both the intelligence and transparency of the system. Integrating Deep Q-Networks or policy-gradient methods could enable dynamic learning across multiple assets and time resolutions. Adding cost-sensitive reward functions and position-sizing mechanisms would further improve realism and profitability. Moreover, deploying adaptive explanation engines that personalise tone, depth, and language style can elevate the user experience from static explanations to conversational, mentor-like interactions.

A broader user study, ideally involving diverse investor demographics, would also strengthen empirical insight into how explainability affects financial decision-making behaviour.

Broader Implications

Beyond its technical scope, the Financial Advisor Bot underscores a broader theme—the convergence of AI

ethics, transparency, and user empowerment. As algorithmic systems increasingly shape financial decisions, ensuring that users can understand and question those systems becomes a societal necessity. This project contributes to that vision by showing that high-performance algorithms can also be interpretable, educational, and accessible.

Closing Statement

The Financial Advisor Bot fulfils its aim of merging machine intelligence with human understanding. It stands not merely as a trading prototype but as a proof of concept for how explainable AI can transform complex decision systems into tools of empowerment. With its balance of technical depth, user experience, and interpretability, the project provides a solid foundation for future work in explainable financial technology—paving the way for a new generation of intelligent, transparent, and inclusive digital advisors.

References

- Brock, W., Lakonishok, J., & LeBaron, B. (1992). *Simple technical trading rules and the stochastic properties of stock returns*. *Journal of Finance*, 47(5), 1731–1764.
- Chen, Y., Zhang, L., & Wang, P. (2021). *Financial text summarization using GPT-style large language models*. *Proceedings of the International Conference on Computational Finance*, 88–96.
- Freqtrade. (2024). *Freqtrade open-source crypto trading bot*. Retrieved from <https://www.freqtrade.io>
- Haque, A., & Chen, X. (2022). *Template-based natural language generation for financial explanations: A hybrid approach*. *Journal of Intelligent Information Systems*, 58(3), 411–430.
- Jiang, Z., Xu, D., & Liang, J. (2017). *A deep reinforcement learning framework for the financial portfolio management problem*. *arXiv preprint arXiv:1706.10059*.
- Krause, A., Perer, A., & Ng, K. (2016). *Interpretable machine learning for fraud detection: Generating natural-language explanations of model decisions*. *Proceedings of the IEEE Symposium on Computational Intelligence*, 182–189.
- Li, X., Yang, J., & Zhang, Q. (2009). *A reinforcement learning framework for automated stock trading*. *Expert Systems with Applications*, 36(4), 7310–7319.
- Moody, J., & Saffell, M. (2001). *Learning to trade via direct reinforcement*. *IEEE Transactions on Neural Networks*, 12(4), 875–889.
- Neely, C., Weller, P., & Ulrich, J. (2009). *The adaptive markets hypothesis: Evidence from the foreign exchange market*. *Journal of Financial and Quantitative Analysis*, 44(2), 467–488.
- Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). *Predicting stock and stock price index movement using trend deterministic data mining techniques*. *Expert Systems with Applications*, 42(1), 259–268.
- QuantConnect. (2024). *Algorithmic trading and quantitative finance platform*. Retrieved from <https://www.quantconnect.com>
- Backtrader. (2023). *Backtrader: Python backtesting and trading framework*. Retrieved from <https://www.backtrader.com>
- Yahoo Finance. (2025). *Financial market data API*. Retrieved from <https://finance.yahoo.com>
- Robinhood Markets Inc. (2025). *Robinhood investing platform*. Retrieved from <https://robinhood.com>
- eToro Group Ltd. (2025). *Social trading and investment platform*. Retrieved from <https://www.etoro.com>
- Wealthfront Inc. (2025). *Automated investment management platform*. Retrieved from <https://www.wealthfront.com>
- Betterment LLC. (2025). *Automated portfolio management service*. Retrieved from <https://www.betterment.com>
- Alpaca Markets. (2025). *Alpaca API documentation*. Retrieved from <https://alpaca.markets>
- Ollama. (2025). *Local large language model inference engine*. Retrieved from <https://ollama.ai>